# Start extending your agent

Start with one change you can explain and test. You do not need to understand every supplied file before building a project.

The basic loop is: **request → agent chooses a tool → Python does the action → result returns to the agent → answer**. Instructions guide choices; tool code determines what an action actually does.

## Choose a starting point

| I want to… | Start here | Try this first |
|---|---|---|
| Change the agent's behavior | `INSTRUCTIONS` in agent.py | Ask it to explain one disagreement between sources |
| Add an action | tools.py, then `TOOLS` in `agent.py` | Adapt `save_recommendation` to save a research question |
| Test my action without a model | try_tools.py | Change an input and predict the saved file |
| Explore memory | `prepare_input` in `agent.py` | Save a note, then explicitly supply it on a later turn |
| Inspect or edit plans | plans.py | Load a plan and print its steps |
| Build a UI | `open_lab` and `session.run` below | Display an answer and the changed filenames |
| Use multiple agents | project_examples.py | Pass a researcher's answer to a critic |
| Evaluate my project | `evaluation_example` in `project_examples.py` | Run fixed tasks and compare outputs with a rubric |

For a new tool, copy a short decorated function in `tools.py`, change its name and action, import it in `agent.py`, and add it to `TOOLS`. Try it in `try_tools.py` before asking the model to use it. Explain its input, action, and returned result. Restart the terminal agent after editing Python files.

## Call the agent from your own program

Save this as `my_project.py` beside `agent.py`. It uses your configured lab key and makes a paid model call. Run it with `python my_project.py`.

```python
from lab_support import open_lab

with open_lab() as lab:
    session = lab.session()
    result = session.run("Read the interviews and suggest one research question.")
    print(result.status)       # completed, incomplete, failed, or empty
    print(result.answer)
    print(result.artifacts)    # changed filenames, not file contents
    print(result.error)        # explanation if the turn failed
```

Reuse `session` for follow-up messages. A new session starts with an empty conversation; saved files still exist. `lab.session(root=another_folder)` selects different task files while keeping the same connection and spending ledger. Put that task's source files in its `data/` folder.

For a UI, pass `on_event=my_callback` to `lab.session`. The callback receives dictionaries such as `tool_started`, `tool_completed`, `tool_failed`, and `plan_changed`, with run and agent identifiers. `result.events` retains the records; `result.to_dict()` provides a serializable result for experiments. These records can contain source text, so choose what to show or save.

## Work with a plan without asking the model

Use the same validation as the agent tool. Run this between turns, from a script in the lab folder:

```python
from file_access import FileAccess
from plans import load_plan, save_plan
```

```
access = FileAccess(".", emit=lambda text: None)
plan = load_plan(access)
if plan is not None:
    for step in plan.steps:
        print(step.id, step.status, step.description)
    plan.goal = "Focus on small businesses"
    plan.revision_note = "The user narrowed the audience."
    save_plan(plan, access)
```

`runtime/plan.json` stores the fields; `runtime/plan.md` is the readable view. Use `save_plan` to update both. A Markdown-only plan from an older lab needs an explicit conversion: read it, construct its goal/steps/revision note, and save through `save_plan`. The loader reports legacy or inconsistent files.

Editing a plan does not automatically tell the agent about the edit. Extend `prepare_input(history, user_input, files)` in `agent.py` to supply selected information, or explicitly ask the agent to reread the plan. The same distinction matters for memory: **saved information and information currently given to the model are different**. Choose what to include; do not paste every file into every turn. A plan records intended work; it does not enforce execution order.

## Try a workflow or evaluation script

Run `python project_examples.py`. It uses scripted model replies and temporary files, requires no key, and makes no network calls. Read the two short functions:

- `workflow_example` gives two agents separate conversations and explicitly passes an answer between them. They share files; use separate workspace roots if they need separate plans or notes.
- `evaluation_example` starts each task with fresh files and history, records its configuration and result, and keeps accounting shared across trials.

These examples check the program connections, not the agent's intelligence. For a live experiment, use `open_lab()` in place of the offline connection, choose a small task set and rubric, and inspect actions as well as answers. Repeat tasks when assessing reliability. Failed runs and unknown usage belong in your results too. Never reset `.lab-state/` to start a new experiment.

## When your project needs more control

The supplied implementation is in lab_support/session.py and lab_support/connection.py. Start by using their public arguments: `agent_factory`, `runner`, `input_builder`, `on_event`, and `agent_id`. Your factory receives the configured model and returns an SDK agent. Your runner receives `(agent, messages, run_config)` and returns its SDK result. Keep those supplied model and run settings when using the lab budget controls.

For an application with an event loop, use `await session.run_async(message)`. It runs the same code on a worker thread; deliver event callbacks to your UI's own thread or queue. Own `open_lab()` in synchronous setup/teardown code and finish all runs before closing it. Cancellation waits for the running turn; this interface does not pause or interrupt the agent mid-task.

Calls sharing a connection run sequentially. Parallel execution, approval interfaces, automatic memory retrieval, and automated scoring remain extensions you can design for your project. A small, well-evaluated change is a useful project contribution.